Prerequisites and objectives

- Prerequisites are Chapter 1's ADTs, invariants, and cost models, together with C++ object lifetimes.
- Define the Array ADT by its index domain. Derive multidimensional address formulas and explain the limits of contiguous storage.
- Trace sparse transpose and string matching. Distinguish characters from encoding units.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The Array ADT index contract

- A one-dimensional array maps a fixed index domain $\{0,\ldots,n-1\}$ to values. Both `get(i)` and `set(i,x)` require $0 \le i < n$.
- Resizing is an extra capability of a dynamic sequence. It is not required by a fixed-length Array ADT.
- The postcondition of `set(i,x)` changes the value at i to x and preserves every other position.
- Contiguous memory is a representation, not the mathematical ADT definition. Segmented storage can offer indexed semantics with different costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ representation and lifetime

- `std::array<T,N>` has a compile-time fixed length. `std::vector<T>` can change length at runtime.
- Except for the `vector<bool>` specialization, vector elements occupy contiguous storage. `size()` counts existing elements. `capacity()` counts allocated element slots.
- `operator[]` is not a general bounds-checking mechanism. Examples check external indices before access.
- Reallocation invalidates existing element pointers, references, and iterators. `reserve` does not increase `size` to the requested capacity.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Row-major two-dimensional addresses

- For an `R×C` array of fixed-width elements of w bytes, the element offset of zero-based `(i,j)` is `iC+j`.
- With base address b, the byte address is `b+w(iC+j)`. Apply this formula only to valid elements within the same contiguous allocation.
- For the original example `R=3,C=4,w=8`, `(2,1)` has element offset 9 and byte offset 72.
- Rows of `vector<vector<T>>` may be allocated separately. The entire matrix need not be one contiguous block, so the whole-matrix formula does not generally apply.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Traversal order and cache behavior

- In row-major storage, increasing column index j in the inner loop reads adjacent elements.
- Increasing row index i in the inner loop instead gives a stride of C elements. It may reduce reuse of each cache line.
- Both traversals take `0(RC)` RAM operations. Actual times may differ. No cache speed ratio is claimed without measurement.
- Check multiplication bounds before allocating if `R×C` can overflow the index type. A valid logical shape does not guarantee a feasible allocation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COO and CSR sparse representations

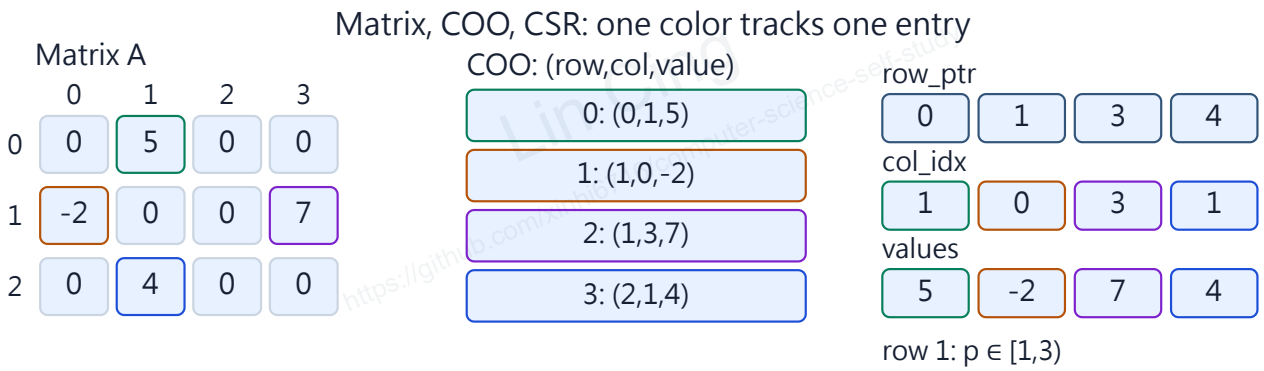
- `nnz=k` counts nonzero entries. COO stores triples `(row,column,value)` and separately retains the matrix dimensions.
- Our COO invariant requires row-major order, valid indices, unique coordinates, and no explicit zeros. Normalize other input by sorting, combining duplicate coordinates, and removing zeros.
- CSR stores `row_ptr`, `col_idx`, and `values`. Row i occupies `[row_ptr[i],row_ptr[i+1])`.
- CSR space is `0(R+k)`. The R term matters when many rows are empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked sparse example

A matrix has 3 rows and 4 columns, with entries `(0,1,5)`, `(1,0,-2)`, `(1,3,7)`, and `(2,1,4)`.

- COO uses the listed order. CSR is `row_ptr=[0,1,3,4]`, `col_idx=[1,0,3,1]`, and `values=[5,-2,7,4]`.
- Row 1 occupies `[1,3)`, giving values −2 and 7 at columns 0 and 3.
- Multiplying by $x=[1,2,3,4]^T$ gives $y=[10,26,8]^T$.



Fast transpose counts columns

1. Count original entries in each column as `count[j]`.
2. Prefix-sum the counts to obtain starting positions `start[j]` for the transposed rows.
3. Scan original COO order. Write `(j,i,v)` at `next[j]` for entry `(i,j,v)`, then increment `next[j]`.
4. Preserve `start` as immutable interval boundaries. Do not overwrite it with the moving cursors.

Time is $O(C+k)$, auxiliary space is $O(C)$, and output space is $O(k)$. A complete output CSR also requires $C+1$ row-pointer positions.

Why fast transpose does not overwrite entries

- Column counts are `[1,2,0,1]`. Prefix starting positions are `[0,1,3,3]`.
- Column output intervals are disjoint, and each length equals its number of entries. Every input entry therefore has one unique output slot.
- Stable distribution in input row order keeps column indices increasing within each transposed row.
- The result is `(0,1,-2)`, `(1,0,5)`, `(1,2,4)`, `(3,1,7)`. An empty column naturally becomes an empty interval.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete COO transpose allocation and cursors

```
transpose(R,C,entries):
    require sorted unique valid nonzero entries
    count=[0]*C
    for (r,c,v) in entries: count[c]+=1
    start=[0]*(C+1)
    for c=0..C-1: start[c+1]=start[c]+count[c]
    next=copy(start[0:C]); out=array(len(entries))
    for (r,c,v) in entries:
        out[next[c]]=(c,r,v); next[c]+=1
    return (C,R,out)
```

For the same 3×4 example, (0,1,5) writes out[1], (1,0,−2) writes out[0], (1,3,7) writes out[3], and (2,1,4) writes out[2]. Start is [0,1,3,3,4]; final next=[1,3,3,4] equals the interval ends. Disjoint intervals with exactly matching write counts prevent overwrites and omissions. Stable distribution by original row preserves sorted columns within each transposed row.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete COO transpose allocation and cursors (continued)

Empty column 2 has interval [3,3). With R=0, C=4, retain result dimensions 4×0. Time is $\Theta(C+k)$, output space $\Theta(k)$, and auxiliary space $\Theta(C)$. Normalizing unsorted or duplicate input adds separate sorting/combination work. Sparse addition also requires equal dimensions, not merely equal nnz.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sparse arithmetic costs

- Add two sorted COO inputs by advancing as in merge sort. Combine equal coordinates and omit a sum of zero.
- For input nonzero counts k_1, k_2, the merge costs $\Theta(k_1+k_2)$, excluding input normalization.
- CSR matrix-vector multiplication $y_i = \Sigma_j A_{ij} x_j$ costs $\Theta(R+k)$, including output initialization.
- Sparse matrix multiplication is not generally $O(k_1+k_2)$. The output may become dense, and accumulating equal coordinates requires additional organization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sparse addition and matrix-vector multiplication

```
add_COO(A,B):
    require same dimensions and canonical row-major entries
    i=0; j=0; out=[]
    while i<len(A) and j<len(B):
        if coord(A[i])<coord(B[j]): out.append(A[i]); i+=1
        else if coord(B[j])<coord(A[i]): out.append(B[j]); j+=1
        else:
            value=A[i].value+B[j].value
            if value!=0: out.append((A[i].row,A[i].col,value))
            i+=1; j+=1
    append remaining A[i:] and B[j:]; return out
CSR_matvec(R,C,row_ptr,col_idx,values,x):
    require len(x)==C
    y=[0]*R
    for r=0..R-1:
        for p=row_ptr[r]..row_ptr[r+1]-1:
            y[r]+=values[p]*x[col_idx[p]]
    return y
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sparse addition and matrix-vector multiplication (continued)

Let B contain(0,1,−5),(2,0,3). Adding it to A cancels(0,1), emits A's(1,0,−2),(1,3,7), then B's(2,0,3), then A's(2,1,4). Four entries remain, not kA+kB=6. For x=[1,2,3,4], row0 gives 5×2=10; row1 accumulates−2 then26; row2 gives 4×2=8.

CSR requires 0≤row_ptr[r]≤row_ptr[r+1]≤k, row_ptr[0]=0, row_ptr[R]=k, and valid columns in[0, C). Equal array lengths alone do not establish these invariants. Arithmetic costs assume fixed-range values and do not cover overflow or arbitrary-precision accumulation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String ADT and encoding levels

- A string is a finite sequence over alphabet Σ. Basic operations include length, concatenation, substring, and comparison.
- `std::string` stores char code units and can contain embedded `\0`. `size()` determines its logical length. C-string termination rules do not always apply.
- A UTF-8 code point can occupy multiple bytes. A user-perceived character can contain multiple code points.
- Matching here compares fixed-size symbols. If symbols are UTF-8 bytes, returned positions are byte positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copying and viewing strings

- With contiguous code units and a stored length, indexed access can cost O(1). Copying a substring of length k takes Θ(k) time and output space.
- Concatenating lengths N and M into an independent result copies Θ(N+M) units. Repeatedly copying the whole accumulated prefix can give quadratic total work.
- A nonowning string view stores a start and length. Slicing can take constant time, but the view does not extend the source lifetime. A released or reallocated source invalidates the view.
- Lexicographic comparison searches for the first differing symbol. The comparison costs $O(\ell + 1)$, where ℓ is the common-prefix length and the additional term covers the end or mismatch check. Equal hashes, pointers, or lengths do not generally replace content equality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naive matching and the KMP prefix function

- Let text length be N and pattern length be M. Comparing at every candidate start has worst-case bound `O(NM)`, for example repeated symbols with a mismatch at the last pattern position.
- Define `π[i]` as the length of the longest proper prefix of `P[0..i]` that is also a suffix.
- After q matched symbols, a mismatch sets `q=π[q-1]` to try a shorter known border. The text position never moves backward.
- Prefix construction and search each have a linear total number of fallback steps. The overall bounds are `O(N+M)` time and `O(M)` auxiliary space.

The fallback chain enumerates possible shorter borders. Any border surviving the mismatch must be a border of the already matched prefix, so skipping other lengths cannot discard a match. Each successful extension increases q by at most one. Each fallback strictly decreases q, so the total number of fallbacks is bounded by prior increases, separately during construction and search.

Worked KMP trace

- For `P=ababaca`, the prefix function is `[0,0,1,2,3,0,1]`.
- After matching `ababa`, the next text symbol is b while the pattern expects c. Fall back from `q=5` to `π[4]=3`.
- Now `P[3]=b` matches that same text symbol, giving `q=4` without rescanning earlier text.
- In `abababaca`, the complete match starts at zero-based position 2. Our interface defines an empty pattern to match at position 0.

Complete KMP construction and search

```
prefix(P):
    pi=[0]*len(P)
    for i=1..len(P)-1:
        q=pi[i-1]
        while q>0 and P[q]!=P[i]: q=pi[q-1]
        if P[q]==P[i]: q+=1
        pi[i]=q
    return pi
first_match(T,P):
    if len(P)==0: return 0
    pi=prefix(P); q=0
    for i=0..len(T)-1:
        while q>0 and P[q]!=T[i]: q=pi[q-1]
        if P[q]==T[i]: q+=1
        if q==len(P): return i-len(P)+1
    return not_found
```

For P=ababaca, construction steps are i=1:b, q=0→0; i=2:a,0→1; i=3:b,1→2; i=4:a,2→3; i=5:c,3→1→0 with no extension; i=6:a,0→1. Thus pi=[0,0,1,2,3,0,1]. Reading T=abababaca gives q=1,2,3,4,5,4,5,6,7. At text index 5, b makes q fall from 5 to 3 before extending to 4. The match ends at i=8 and starts at 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete KMP construction and search (continued)

Construction accesses P[q] only for nonempty P and i≥1. Search handles an empty pattern first and immediately returns when q=M, avoiding P[M]. To enumerate overlapping matches, set q=pi[M−1] after each success instead of clearing it. Fixed-symbol comparison gives O(N+M) total work. Byte-oriented UTF-8 matching returns byte offsets, not grapheme indices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and extension boundaries

- Container properties follow the C++ vector overview. Reallocation rules appear in vector capacity.
- Knuth, Morris, and Pratt published *Fast Pattern Matching in Strings* in 1977. The SIAM publication record supplies the bibliography. This chapter independently explains the prefix-function method rather than reproducing the article.
- Encoding distinctions can be checked against the Unicode UTF FAQ.
- KMP and CSR are explicit teaching extensions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study